

How a tree is drawn

There is no L-system, no branching, no recursion and no imported model. A tree in Šibenik is a **stack of rings lofted into a tube** — one prototype per species, drawn thousands of times by four instanced draw calls. Two functions do all of it.

1 The ring

Every profile in the landscape is generated by one twelve-line function. It walks n points around a circle at height y , modulating the radius as it goes:

SRC/45-TREES.JS:47

```
function vegRing(y, r, seg, jag = 0) {
  const pts = [];
  for (let i = 0; i < seg; i++) {
    const a = (i / seg) * TAU;
    const rr = r * (1 + jag * Math.sin(i * 2.37 + y * 6.1));
    pts.push(new THREE.Vector3(Math.cos(a) * rr, y, Math.sin(a) * rr));
  }
  return pts;
}
```

Which is to say, for $i = 0 \dots n - 1$:

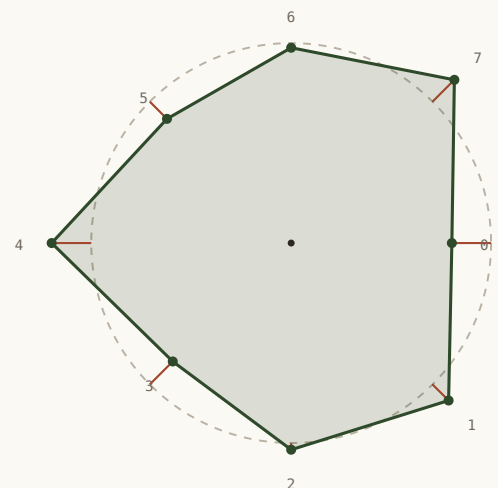
$$\begin{aligned}\theta_i &= \frac{2\pi i}{n} \\ \rho_i &= r(1 + j \sin(2.37i + 6.1y)) \\ P_i &= (\rho_i \cos \theta_i, y, \rho_i \sin \theta_i)\end{aligned}$$

The whole trick is the j term — the `jag` argument. Take it away and you have a lathe: a perfectly circular canopy that the eye reads as a cone of revolution from any angle.

2.37 is deliberately not an integer. If the frequency divided evenly into 2π the wobble would close up around the ring and simply produce a smaller, still symmetric polygon. Because it does not, vertex 0 and vertex n disagree about the radius, and the seam falls somewhere arbitrary.

The 6.1y term re-phases per height. The ring at $y = 0.62$ bulges in different directions from the one at $y = 0.74$, so the canopy leans and lumps down its length instead of being an extruded silhouette.

i	0	1	2	3	4	5	6	7
ρ_i	0.362	0.501	0.465	0.377	0.539	0.395	0.440	0.520
of r	80%	111%	103%	84%	120%	88%	98%	116%



canopy ring at $y = 0.74$, $r = 0.45$, $j = 0.20$, $n = 8$

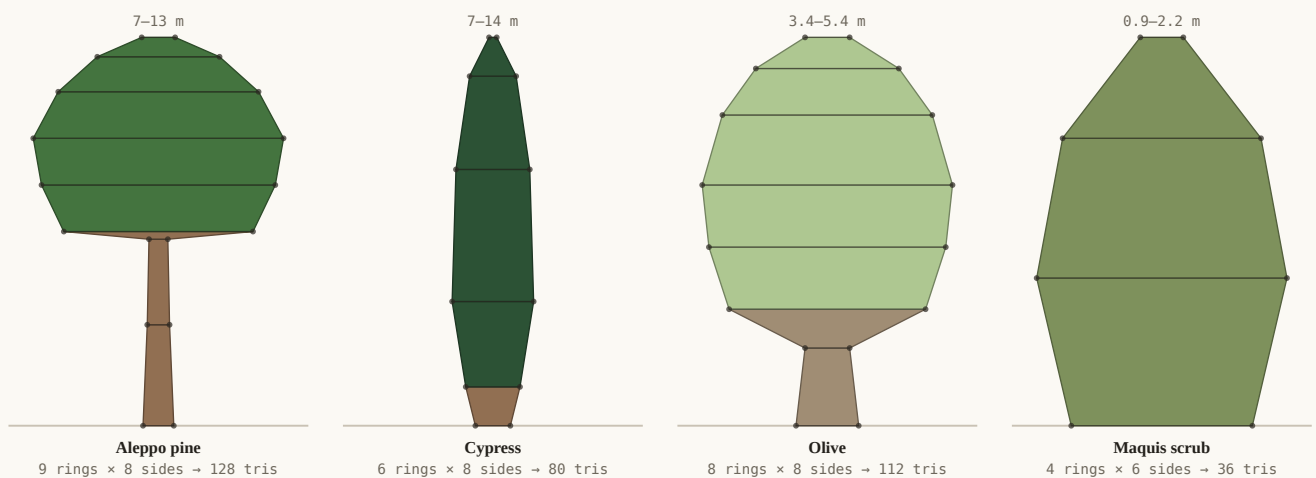
2 The species

A species is then just a table of numbers. An Aleppo pine is nine of them — three thin rings for the bare trunk, then the flat umbrella that reads as Dalmatia from a thousand feet:

SRC/45-TREES.JS:85

```
const pine = vegGeo([
  vegRing(0.00, 0.055, S), vegRing(0.26, 0.040, S), vegRing(0.48, 0.034, S),
  vegRing(0.50, 0.34, S, 0.22), vegRing(0.62, 0.42, S, 0.21),
  vegRing(0.74, 0.45, S, 0.20), vegRing(0.86, 0.36, S, 0.19),
  vegRing(0.95, 0.22, S, 0.16), vegRing(1.00, 0.06, S),
], S, 3, bark, [0.14, 0.24, 0.13]);
```

Everything is normalised to height 1, so y runs $0 \rightarrow 1$ and radii are in the same units. $S = 8$ is the side count. `split = 3` means rings 0-2 are painted bark and everything above is leaf — as *vertex colours*, so a single instanced draw carries both materials without a second pass.



EVERY PROFILE IN THE GAME, DRAWN FROM ITS OWN RING TABLE. TICKS MARK THE RINGS; BROWN BANDS ARE BELOW THE SPLIT.

Skinning the stack is the other half. `loft()` emits the quad grid between consecutive rings — two triangles per side per gap, wound consistently:

SRC/30-MATERIAL.JS:180

```
const a = at(r, p), b = at(r, p + 1),
      c = at(r + 1, p + 1), d = at(r + 1, p);
idx.push(a, b, c, a, c, d);
```

$$T = 2(R - 1)n$$

for R rings of n sides. A pine is 128 triangles, a cypress 80, an olive 112, a maquis bush 36. **356 triangles is the entire flora of Šibenik** — everything else is instancing.

Eight sides, not five. A pentagonal cross-section is what made the hillside read as faceted from low down; eight is the point where a canopy stops having corners. Since the landscape is four draw calls whatever the count, that is the entire cost.

3 The individual

The prototype is identical for every pine on the coast. All the variety is per-instance, from a `mulberry32` stream seeded on the 512 m tile coordinate — so the same tree is in the same place every time you fly back over it.

```
const h = lo + rng() * (hi - lo);
out[pick].push({
  x, y, z, h,
  w: h * wide * (0.78 + rng() * 0.44) / ((lo + hi) * 0.5),
  yaw: rng() * TAU,
  tint: 0.80 + rng() * 0.34,
  warm: rng() * 0.16,
});
```

$$h = h_{lo} + \xi(h_{hi} - h_{lo})$$

$$w = \frac{hk(0.78 + 0.44\xi)}{\frac{1}{2}(h_{lo} + h_{hi})}$$

with ξ a fresh uniform draw on $[0, 1)$, and k the species' canopy aspect (1.9 for pine, 1.0 for cypress). Note that w is proportional to *this* tree's h rather than drawn independently: a tall pine is a big pine, and you never get a spindly giant standing next to a fat dwarf.

PER FRAME

Two more multipliers are applied when the visible set is repacked, roughly twice a second and whenever the camera crosses a tile boundary:

```
const fade = 1 - sat((Math.sqrt(d2) - R * 0.80) / (R * 0.20));
if (fade < 0.02) continue;

const char = fire ? fire.charAt(t.x, t.z) : 0;
const alight = fire ? fire.intensityAt(t.x, t.z) : 0;
const shrink = (1 - 0.40 * char) * (0.35 + 0.65 * fade);
```

$$u = \frac{d - 0.80R}{0.20R}$$

$$f = 1 - \text{sat}(u)$$

$$s = (1 - 0.40c)(0.35 + 0.65f)$$

Here d is distance to camera and R the 2200 m draw radius, so u is how far you are into the outer fifth of it — 0 at $0.80R$, 1 at the edge. `sat` clamps it, and f sinks the tree into the ground across that band. A hard cutoff draws a visible ring of forest around the hillside and the eye finds it instantly; a fade does not.

The second, s , is the fire reading. c is the char value from the automaton at that tree's position — the same grid the fire simulation burns — so a tree standing in a burning cell shrinks 40 % and its instance colour is driven from green to a black stick. The flame itself is somebody else's problem, drawn by `38-flames.js`.